RAPPORT DE STAGE · FIN DE FORMATION 2025–2026

Développement Full-Stack & Coordination Agile

Chez Landmark Agency — Oujda

Migration Next.js · Architecture en 6 Couches · CI/CD · DevOps
Sécurité JWT · Tests Vitest · Scrum Master · Aligno & SikBeauty

STAGIAIRE
Badr Eddine Slaoui

ENCADRANT ENTREPRISE
M. LATRACH Mohammed

FORMATEUR RÉFÉRENT
M. Saad Shaimer (YouCode)

CME
Mme Kaoutar Darroufi

PÉRIODE
7 Mai — 7 Juillet 2026

ÉTABLISSEMENT
YouCode Safi · 2025–2026

RÉSUMÉ

Ce rapport de stage présente le bilan de deux mois d'immersion professionnelle au sein de **Landmark Agency** (Oujda), du 7 mai au 7 juillet 2026. Au cours de cette période, j'ai exercé un double rôle de **Scrum Master** et de **Développeur Full-Stack Lead** sur deux projets de natures différentes mais complémentaires.

Le premier projet, **SikBeauty**, est une marketplace e-commerce de cosmétiques coréens. Ma mission a consisté à orchestrer la migration technique d'une Single Page Application React vers une architecture Next.js avec Server-Side Rendering, afin d'améliorer drastiquement l'indexation SEO et les performances. J'ai également déployé une infrastructure DevOps complète comprenant Docker, un pipeline CI/CD GitHub Actions et un hébergement sur VPS Hostinger.

Le second projet, **Aligno**, est un écosystème de productivité complet développé entièrement de zéro, intégrant un tableau Kanban collaboratif en temps réel, un gestionnaire de temps par blueprints et un module de finances personnelles. J'ai conçu et implémenté l'architecture backend en 6 couches (Route Handler, Controller, Service, Repository, DAO, DTO/Mapper), et développé le module d'authentification sécurisé complet, avec validation des e-mails via Resend, gestion des tokens JWT, cookies HttpOnly et une suite de 35 tests d'intégration avec Vitest.

En tant que Scrum Master, j'ai planifié et coordonné 4 sprints hebdomadaires, géré le backlog Jira, structuré le workflow Git, réalisé les revues de code et accompagné techniquement mes 4 collègues développeurs.

Mots-clés : Nuxt 4, Vue 3, Next.js, TypeScript, Prisma ORM, PostgreSQL, Docker, GitHub Actions, Scrum Master, JWT, bcrypt, Zod, Vitest, SEO, DevOps, Architecture en Couches.

ABSTRACT

This internship report documents two months of professional immersion at **Landmark Agency** (Oujda, Morocco), from May 7 to July 7, 2026. During this period, I held a dual role as **Scrum Master** and **Lead Full-Stack Developer** on two complementary projects.

The first project, **SikBeauty**, is a Korean cosmetics e-commerce marketplace. I led the technical migration from a React SPA to a Next.js SSR architecture, dramatically improving

SEO indexation and page performance. I also set up a complete DevOps infrastructure including Docker, a GitHub Actions CI/CD pipeline, and VPS deployment on Hostinger.

The second project, **Aligno**, is a comprehensive productivity ecosystem built from scratch, featuring a real-time collaborative Kanban board, a time management module with daily blueprints, and a personal finance tracker. I designed and implemented a 6-layer clean backend architecture and built the entire secure authentication module with email verification via Resend, JWT tokens, HttpOnly cookies, and a suite of 35 integration tests using Vitest.

Keywords: Nuxt 4, Vue 3, Next.js, TypeScript, Prisma ORM, PostgreSQL, Docker, GitHub Actions, Scrum Master, JWT, bcrypt, Zod, Vitest, SEO, DevOps, Layered Architecture.

REMERCIEMENTS

À l'issue de ce stage de fin de formation, il m'est particulièrement agréable de pouvoir exprimer ma reconnaissance à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de cette expérience professionnelle marquante.

Je tiens tout d'abord à adresser mes remerciements les plus sincères à **M. LATRACH Mohammed**, encadrant de stage chez Landmark Agency, pour la confiance qu'il m'a accordée en me confiant des responsabilités techniques de haut niveau dès le premier jour. Ses retours réguliers, ses orientations précises et sa vision professionnelle exigeante ont constitué une source d'apprentissage permanente et ont profondément influencé ma manière d'aborder le développement logiciel.

Je remercie également **M. Saad Shaimer**, formateur référent au sein de YouCode Safi, dont l'enseignement rigoureux et la pédagogie bienveillante ont constitué le socle de mes compétences techniques. Ses évaluations régulières et ses orientations judicieuses durant toute la durée de la formation ont guidé mon évolution.

Ma gratitude va aussi à **Mme Kaoutar Darroufi**, CME, pour son suivi attentif et humain, ses encouragements constants et l'intérêt sincère qu'elle a toujours porté au développement personnel et professionnel des apprenants.

Je souhaite enfin remercier chaleureusement mes collègues de stage, sans qui la réalisation de ces deux projets d'envergure n'aurait pas été possible :

- **Mouna Saoui** — pour sa rigueur, son sens du détail et son investissement constant dans chacune de ses contributions.
- **Brahim El Habchi** — pour sa créativité, sa réactivité et sa capacité à concevoir des solutions techniques élégantes.
- **Yassine Nmila** — pour sa polyvalence, son esprit d'initiative et sa contribution précieuse à chaque sprint.
- **Khadija El Haddad** — pour son professionnalisme, sa persévérance et la qualité irréprochable de son travail.

À tous, merci.

SOMMAIRE

Résumé / Abstract ..... **II**

Remerciements ..... **III**

Introduction Générale **1**

Chapitre 1 : Contexte et Environnement de Stage **3**

1.1 Présentation de Landmark Agency 3

1.2 Présentation de YouCode Safi 5

1.3 Présentation de l'équipe de stage 6

Chapitre 2 : Méthodes de Travail et Rôle de Scrum Master **7**

2.1 La méthodologie Agile Scrum 7

2.2 Rôle et responsabilités du Scrum Master 8

2.3 Gestion du Git Flow et revues de code 10

Chapitre 3 : Architecture Globale des Projets **11**

3.1 Topologie système d'Aligno 11

3.2 Architecture Backend en 6 couches 12

3.3 Diagrammes UML et ERD 13

3.4 Déploiement et Infrastructure 15

Chapitre 4 : Projet SikBeauty — Migration et Industrialisation **16**

4.1 Présentation et contexte 16

4.2 Migration React vers Next.js et optimisation SSR/SEO 17

4.3 DevOps, Docker et CI/CD GitHub Actions 20

4.4 Déploiement sur VPS Hostinger 22

4.5 Contributions Backend 23

Chapitre 5 : Projet Aligno — Conception et Développement from Scratch **24**

5.1 Présentation d'Aligno 24

5.2 Initialisation du projet et configuration 25

5.3 Développement Backend du module Authentification 26

5.4 Tests unitaires et d'intégration avec Vitest 31

5.5 Planification Frontend sur Jira 33

5.6 Développement Frontend — Module Auth (Nuxt 4) 34

5.7 Phase de finalisation du projet 37

Chapitre 6 : Présentation des Technologies **39**

Chapitre 7 : Difficultés Rencontrées — Études de Cas **44**

Chapitre 8 : Bilan des Compétences Acquises **47**

Chapitre 9 : Résultats Obtenus **50**

Conclusion Générale **52**

Annexes **54**

LISTE DES FIGURES

Fig. 1 : Topologie générale du système distribué Aligno 11

Fig. 2 : Architecture Backend en 6 couches (layered architecture) 12

Fig. 3 : Diagramme de cas d'utilisation (Use Case) — Module Auth 13

Fig. 4 : Diagramme de séquence — Inscription et validation e-mail 14

Fig. 5 : Diagramme Entité-Association (ERD) simplifié d'Aligno 14

Fig. 6 : Diagramme de déploiement — Pipeline CI/CD et VPS 15

Fig. 7 : Schéma du Git Flow appliqué aux deux projets 10

Fig. 8 : Structure des répertoires du projet Aligno ..... Annexe A

Fig. 9 : Capture — Tableau Kanban (Jira) en Sprint 3 ..... Annexe B

Fig. 10 : Capture — Rapport Lighthouse SEO de SikBeauty ..... Annexe C

Fig. 11 : Capture — Pipeline GitHub Actions (CI/CD) ..... Annexe D

LISTE DES TABLEAUX

Tableau 1 : Équipe projet et répartition des rôles 6

Tableau 2 : Cérémonies Scrum et planning hebdomadaire 9

Tableau 3 : Comparatif React SPA vs Next.js SSR (performances) 18

Tableau 4 : Métriques SEO Lighthouse avant/après migration 19

Tableau 5 : Variables d'environnement du projet Aligno 25

Tableau 6 : Registre des endpoints API du module Authentification 28

Tableau 7 : Règles de validation Zod par endpoint 30

Tableau 8 : Contrôles de sécurité implémentés dans Aligno 31

Tableau 9 : Couverture des tests Vitest (35 tests) 32

Tableau 10 : Fiches technologiques — Stack Frontend 39

Tableau 11 : Fiches technologiques — Stack Backend et Temps Réel 41

Tableau 12 : Fiches technologiques — DevOps et Qualité 43

Tableau 13 : Bilan comparatif compétences techniques Avant/Après 48

Tableau 14 : Bilan compétences transversales et managériales 49

Tableau 15 : Récapitulatif des indicateurs clés du stage 50

INTRODUCTION GÉNÉRALE

Le secteur du développement logiciel est aujourd'hui marqué par une transformation continue, alimentée par l'émergence de frameworks meta performants, la démocratisation des architectures microservices et l'adoption généralisée des pratiques DevOps au sein des organisations numériques. Dans ce contexte d'évolution rapide, la formation théorique dispensée en école doit impérativement se confronter aux réalités du terrain, aux contraintes de production et aux exigences de la livraison continue de valeur logicielle.

C'est dans cette perspective que s'inscrit mon stage de fin d'études, effectué au sein de **Landmark Agency**, une agence numérique innovante basée à Oujda, du 7 mai au 7 juillet 2026. Ce stage de deux mois a représenté l'opportunité rare et exigeante d'occuper simultanément un rôle de coordination agile et de développement technique de haut niveau, sur deux projets distincts aux natures complémentaires.

D'un côté, le projet **SikBeauty** a demandé une intervention d'industrialisation : transformer une application React existante en plateforme Next.js performante, outillée d'un pipeline CI/CD robuste et d'une infrastructure Docker déployée en production. De l'autre, **Aligno** a représenté le défi de concevoir et de construire from scratch un écosystème logiciel complet — un système de productivité intégrant gestion de projets Kanban, planification du temps et suivi des finances personnelles — avec une architecture rigoureuse, des tests automatisés et une interface soignée.

Ma double casquette de **Scrum Master** et de **Développeur Full-Stack Lead** m'a amené à opérer à plusieurs niveaux : planification des sprints et gestion du backlog Jira, revues de code et coordination d'une équipe de cinq personnes, conception architecturale, développement de fonctionnalités critiques, écriture de tests d'intégration et optimisation des performances en production.

Ce rapport constitue le compte-rendu académique et professionnel de cette expérience. Il est structuré de manière à refléter la progression logique de notre travail : après une présentation du contexte (Landmark Agency, YouCode), des méthodes de travail (Scrum, Git Flow), et de l'architecture technique globale, nous détaillons nos contributions sur chacun des deux projets, puis nous dressons un bilan des compétences acquises, des difficultés surmontées et des résultats obtenus.

CHAPITRE 1 — CONTEXTE ET ENVIRONNEMENT DE STAGE

1.1 Présentation de Landmark Agency

Fondée à Oujda dans le nord-est du Maroc, **Landmark Agency** est une agence de services numériques spécialisée dans la création d'applications web et mobiles sur mesure, le conseil en architecture logicielle et l'accompagnement à la transformation digitale de ses clients. L'agence se distingue par une capacité à prendre en charge des projets complexes de bout en bout, depuis la conception fonctionnelle jusqu'au déploiement en production, en passant par la définition de l'architecture technique et la mise en place des processus qualité.

La culture d'entreprise de Landmark Agency repose sur trois piliers fondamentaux : **l'exigence technique**, **la formation continue** et **l'adoption proactive de technologies modernes**. Les équipes évoluent dans un environnement de travail stimulant, où la veille technologique est encouragée et où chaque projet constitue une opportunité d'explorer de nouveaux paradigmes. La stack technologique de l'agence est résolument orientée JavaScript : Node.js, React/Next.js, Nuxt/Vue.js, TypeScript, avec une expertise forte en DevOps (Docker, GitHub Actions, VPS).

La méthodologie de développement adoptée est le **Scrum Agile**, avec des sprints hebdomadaires structurés autour des cérémonies classiques (Planning, Daily, Review, Retrospective). Cette organisation permet une livraison rapide et continue de valeur, tout en maintenant une visibilité claire sur l'avancement de chaque projet. Les outils utilisés sont Jira pour la gestion de projet, GitHub pour le versioning et la collaboration, et Slack pour la communication interne.

1.2 Présentation de YouCode Safi

YouCode Safi est une école de formation aux métiers du numérique, fruit d'un partenariat pédagogique avec le réseau Simplon. Sa pédagogie repose sur l'apprentissage par projets (*learning by doing*), sans cours magistraux traditionnels : les apprenants progressent en résolvant des problèmes réels, en travaillant en équipe et en se confrontant quotidiennement aux technologies du marché. Cette approche développe chez les stagiaires une forte capacité d'adaptation, d'auto-apprentissage et de collaboration professionnelle.

L'objectif de YouCode est de former des développeurs Full-Stack opérationnels, capables de s'intégrer immédiatement dans une équipe professionnelle. La formation aborde les fondamentaux du développement web (HTML, CSS, JavaScript), puis progresse vers des technologies avancées (Node.js, React/Vue, bases de données relationnelles, DevOps), tout en intégrant des modules de développement personnel et de soft skills.

1.3 Présentation de l'équipe de stage

L'équipe projet était constituée de cinq développeurs stagiaires, coordonnés par l'encadrant de l'entreprise. La répartition des rôles a été définie dès le premier sprint, lors d'une session de planning collective :

NOM & PRÉNOM	RÔLE PRINCIPAL	MODULES PRINCIPAUX CONTRIBUÉS
M. LATRACH Mohammed	Encadrant / Product Owner	Supervision, arbitrage fonctionnel et technique, revue finale.
Badr Eddine Slaoui	Scrum Master & Dev Full-Stack Lead	Auth (Backend + Frontend), DevOps, SEO, Architecture, Tests, Git.
Brahim El Habchi	Développeur Full-Stack	Module Kanban (Backend + Frontend), intégration Socket.IO.
Yassine Nmila	Développeur Full-Stack	Module Wallet (Backend), gestion des transactions et budgets.
Khadija El Haddad	Développeuse Full-Stack	Module Time Management (Frontend), composants UI partagés.
Mouna Saoui	Développeuse Full-Stack	Module Wallet (Frontend), ApexCharts, Dashboard analytique.

Tableau 1 : Équipe projet et répartition des rôles

CHAPITRE 2 — MÉTHODES DE TRAVAIL ET RÔLE DE SCRUM MASTER

2.1 La méthodologie Agile Scrum

Scrum est un cadre de travail agile fondé sur des itérations courtes, appelées sprints, et sur une livraison incrémentale de valeur. Contrairement aux méthodologies en cascade (*waterfall*), Scrum privilégie l'adaptation continue aux changements et la communication permanente entre les membres de l'équipe. Les trois piliers fondamentaux de Scrum sont la **transparence** (tout le monde partage le même niveau d'information), l'**inspection** (le travail est régulièrement évalué) et l'**adaptation** (le plan est ajusté en fonction des observations).

Nos sprints avaient une durée d'une semaine. Chaque sprint débutait par une session de Sprint Planning où l'équipe sélectionnait les tickets du backlog à traiter, se terminait par une Sprint Review de démonstration des fonctionnalités réalisées, et une Sprint Retrospective d'amélioration continue. Les Daily Meetings quotidiens (15 minutes maximum) permettaient de synchroniser l'équipe et d'identifier immédiatement les blocages.

CÉRÉMONIE	FRÉQUENCE	DURÉE	OBJECTIF
Sprint Planning	Début de sprint (lundi)	1h – 1h30	Sélection des tickets, estimation, attribution des tâches.
Daily Meeting	Chaque matin	15 min max	Synchronisation : fait hier, prévu aujourd'hui, blocages.
Sprint Review	Fin de sprint (vendredi)	30 – 45 min	Démonstration des fonctionnalités à l'encadrant (Product Owner).
Sprint Retrospective	Fin de sprint	30 min	Analyse collective des points forts et axes d'amélioration.

Tableau 2 : Cérémonies Scrum et planning hebdomadaire

2.2 Rôle et responsabilités du Scrum Master

Tout au long du stage, j'ai exercé le rôle de **Scrum Master** sur les deux projets simultanément. Ce rôle est bien plus qu'une simple coordination administrative : il s'agit de

mettre en place et de maintenir un environnement propice à la haute performance d'une équipe agile. Mes responsabilités étaient multiples et entrelacées.

En amont de chaque sprint, j'assurais la **gestion et le raffinage du backlog produit** sur Jira. Cela impliquait de créer les Epics structurant chaque module fonctionnel (Authentification, Kanban, Wallet, Time Management, Dashboard), puis de rédiger des User Stories précises respectant la structure : *"En tant que [rôle], je veux [fonctionnalité] afin de [bénéfice]"*. Chaque User Story était ensuite décomposée en tâches de développement atomiques (moins de 8 heures chacune), accompagnées de critères d'acceptation clairs et testables.

Lors du Sprint Planning, j'animais la session d'estimation en utilisant la technique du **Planning Poker** (adaptée à notre contexte). Chaque développeur estimait la charge de chaque tâche en story points, ce qui permettait d'équilibrer la charge de travail et d'éviter la sur-allocation. J'attribuais ensuite les tickets à chaque membre en fonction de ses compétences, de sa disponibilité et des dépendances entre les tâches.

Au quotidien, j'animais les **Daily Meetings** en garantissant leur concision et leur efficacité. Lorsqu'un développeur signalait un blocage, j'intervenais soit pour apporter une solution technique immédiate, soit pour planifier une session de pair-programming. Je maintenais le tableau Kanban de Jira à jour en temps réel, ce qui offrait une visibilité complète sur l'état du sprint à tout moment.

En fin de sprint, j'organisais les **Sprint Reviews** en préparant la démonstration des fonctionnalités et en coordonnant les présentations des développeurs. Les **Sprint Retrospectives** étaient structurées autour de trois axes : ce qui a bien fonctionné, ce qui doit être amélioré, et les actions concrètes à mettre en place dès le sprint suivant.

2.3 Gestion du Git Flow et revues de code

En tant que Lead technique, j'ai défini et imposé un **Git Flow** rigoureux pour les deux projets. L'arbre des branches était structuré ainsi :

- `main` : branche de production, protégée, accessible uniquement via PR approuvées.
- `dev` : branche d'intégration continue, base de toutes les branches de fonctionnalité.
- `feature/[ticket-id]-[description]` : branches de développement par fonctionnalité.
- `fix/[ticket-id]-[description]` : branches de correction de bugs.
- `chore/[description]` : branches de maintenance (mises à jour de dépendances, configuration).

Toutes les fusions dans `dev` passaient obligatoirement par une **Pull Request** sur GitHub, que je relisais systématiquement. Mes revues de code portaient sur plusieurs dimensions : conformité aux critères d'acceptation de la User Story, respect des standards de codage TypeScript, sécurité (absence de secrets en dur, validation Zod présente), qualité de l'architecture (respect des couches), et passage des tests automatiques dans le pipeline CI.

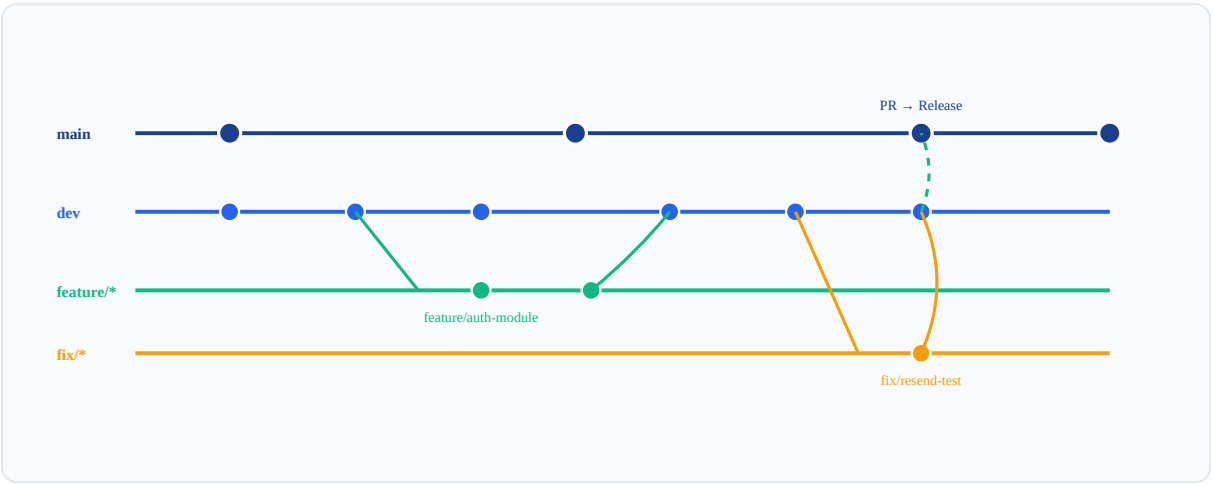


Fig. 7 : Schéma du Git Flow appliqué aux deux projets (main → dev → feature / fix)

CHAPITRE 3 — ARCHITECTURE GLOBALE DES PROJETS

3.1 Topologie système d'Aligno

Aligno est construit sur une topologie de système distribué exploitant pleinement les capacités Full-Stack de Nuxt 4. Le serveur Nitro intégré à Nuxt sert simultanément les pages Vue.js (SSR) et les endpoints REST de l'API. La communication en temps réel pour les fonctionnalités collaboratives (Kanban, notifications) est assurée par Socket.IO, dont le bus de messages est synchronisé entre instances via un adaptateur Redis.

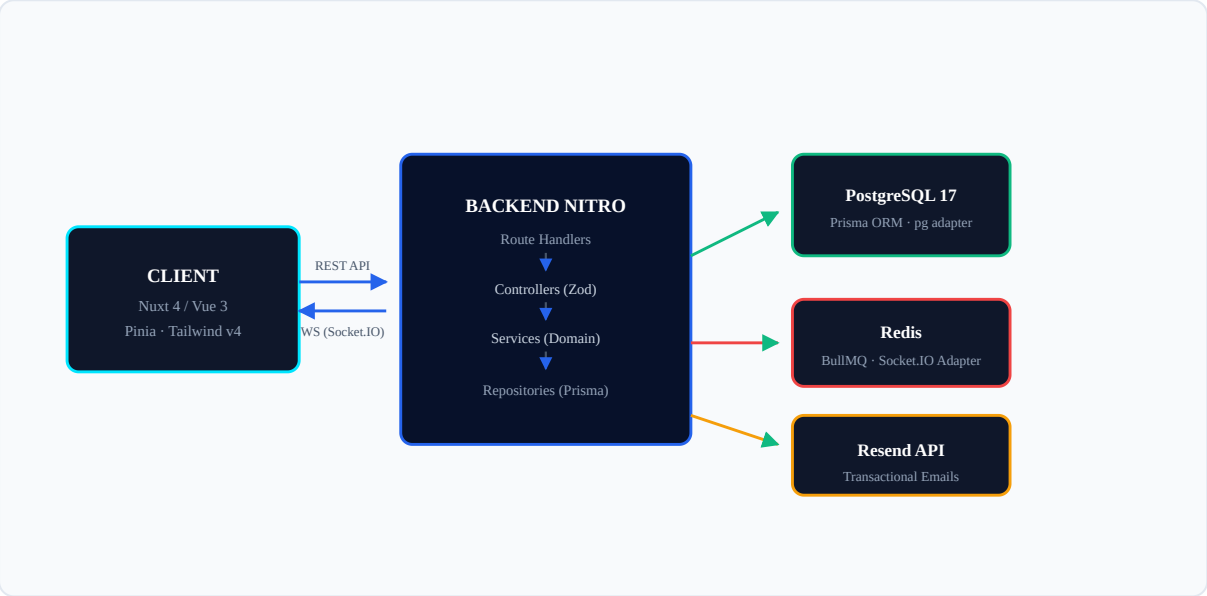


Fig. 1 : Topologie générale du système distribué Aligno

3.2 Architecture Backend en 6 couches

Le backend d'Aligno applique un pattern d'architecture en couches (*Layered Architecture*) strict, inspiré du Domain-Driven Design. Ce choix architectural garantit la séparation des préoccupations, la testabilité unitaire de chaque couche et la maintenabilité à long terme du code. Chaque couche a une responsabilité unique et ne peut communiquer qu'avec les couches adjacentes.

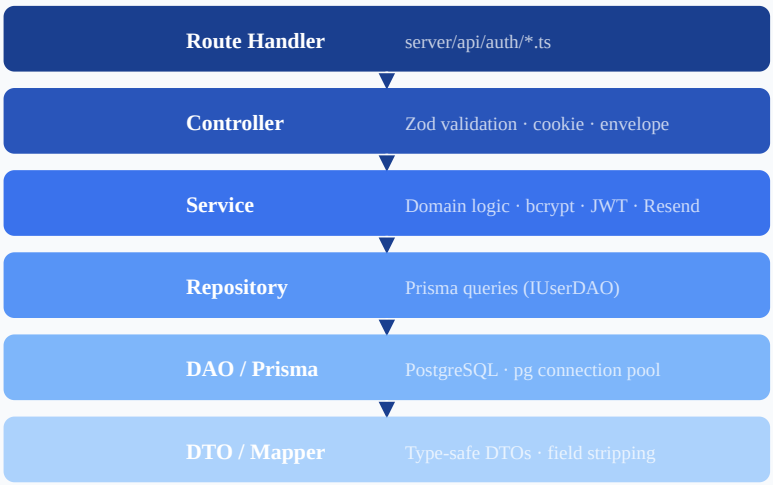


Fig. 2 : Architecture Backend en 6 couches (layered architecture) d'Aligno

3.3 Diagrammes UML et ERD

3.3.1 Diagramme de cas d'utilisation — Module Authentification

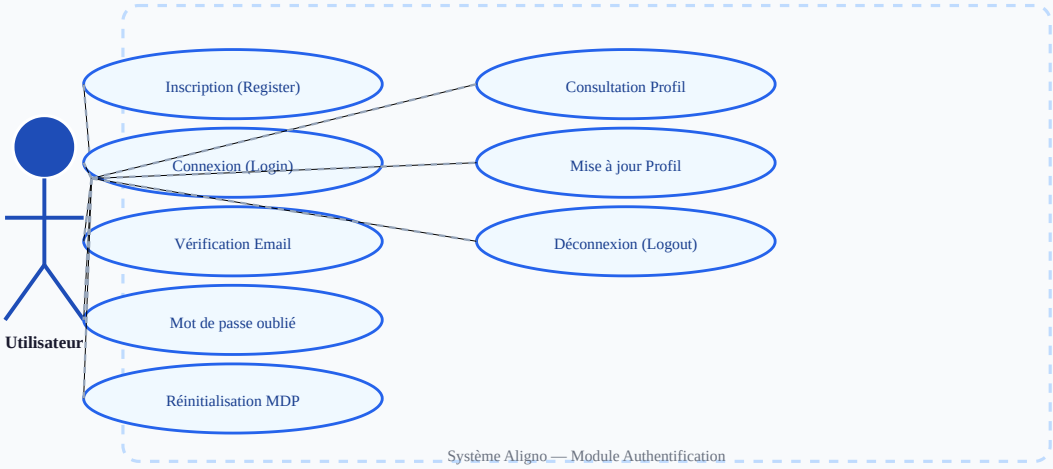


Fig. 3 : Diagramme de cas d'utilisation (Use Case) — Module Authentification d'Aligno

3.3.2 Diagramme de séquence — Inscription et validation e-mail

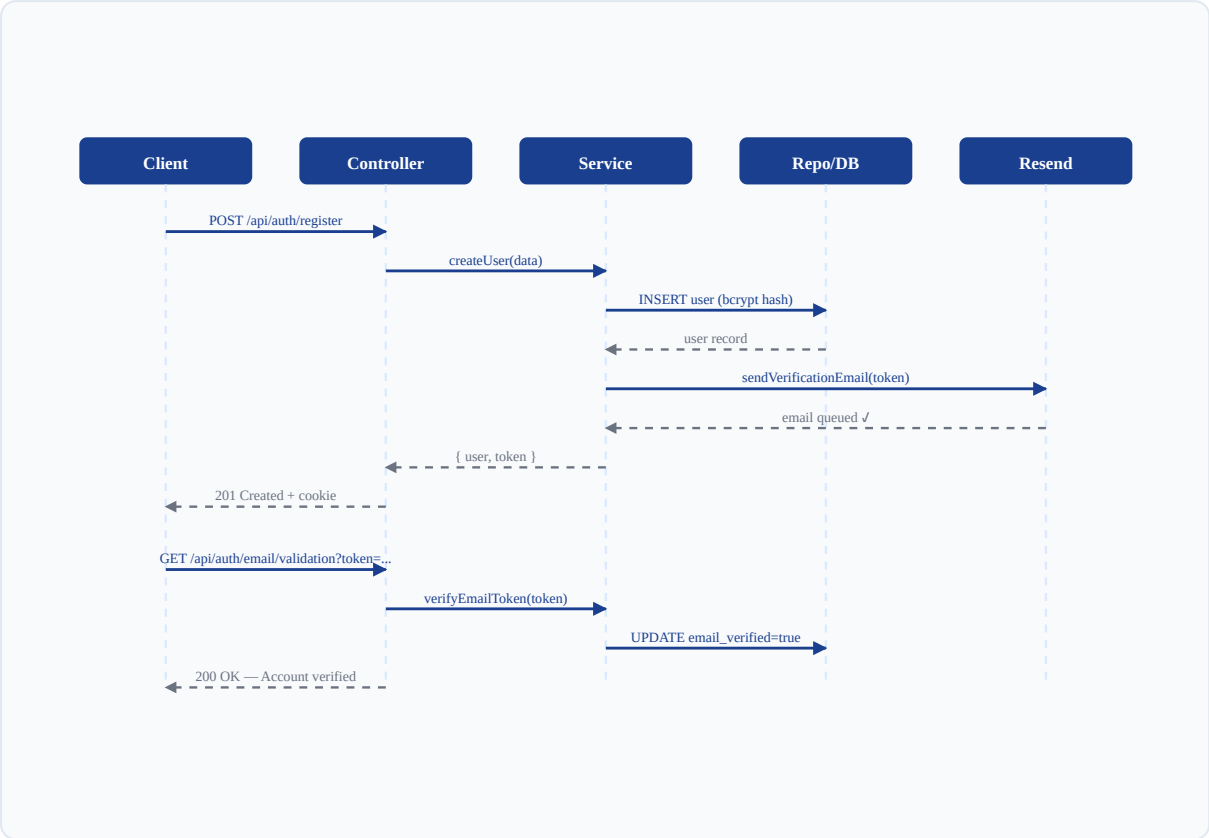


Fig. 4 : Diagramme de séquence UML — Inscription (Register) et validation de l'adresse e-mail via Resend

3.3.3 Diagramme Entité-Association (ERD)

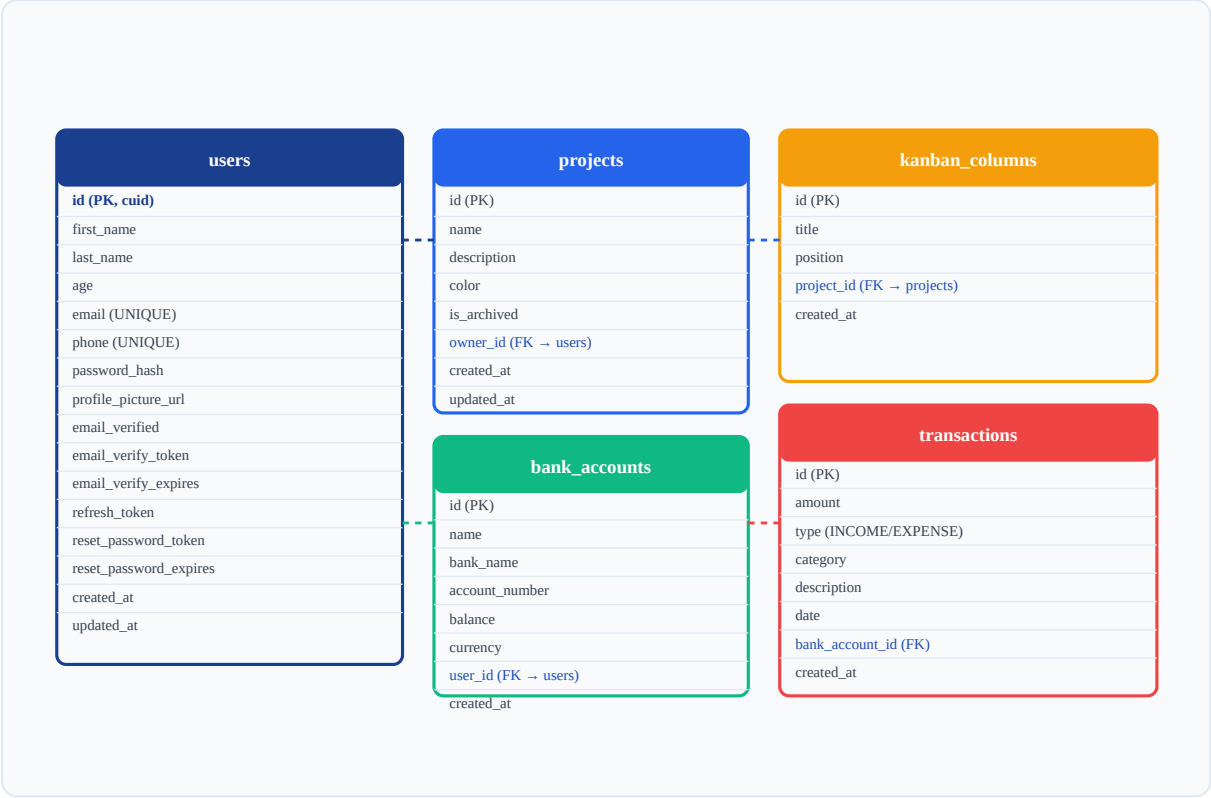


Fig. 5 : Diagramme Entité-Association simplifié d'Aligno (tables principales : users, projects, bank_accounts, kanban_columns, transactions)

3.4 Déploiement et Infrastructure

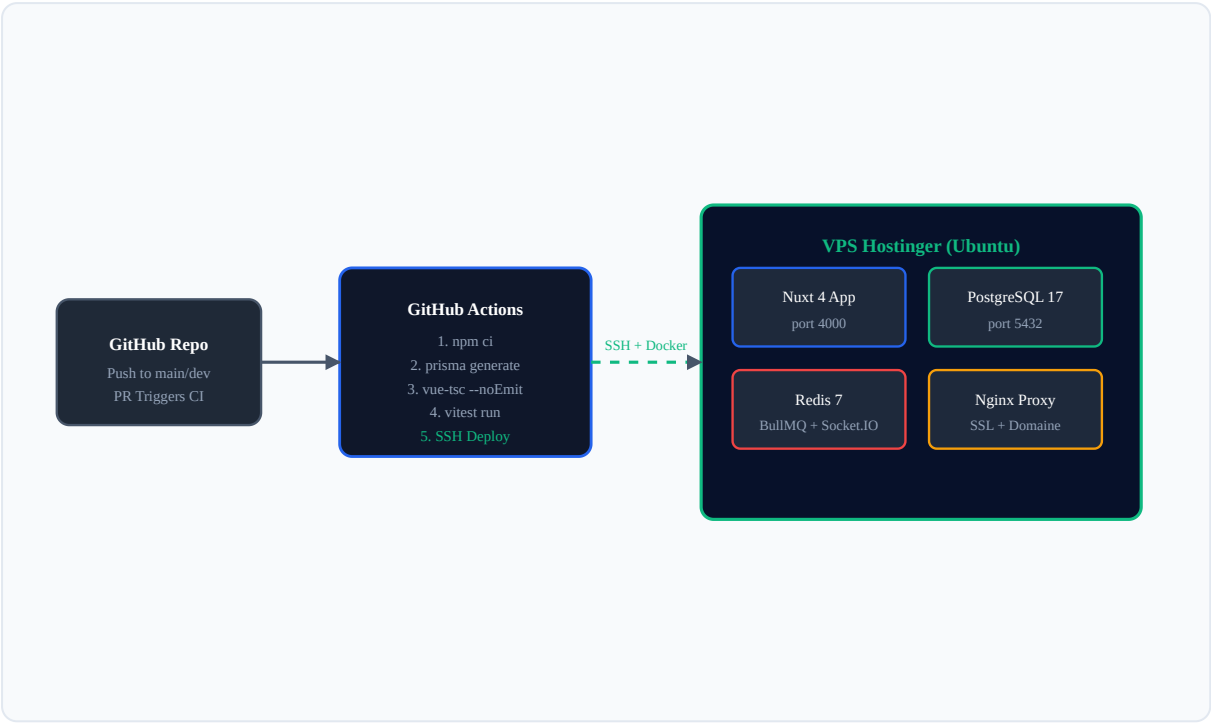


Fig. 6 : Diagramme de déploiement — Pipeline CI/CD GitHub Actions et infrastructure VPS Hostinger

CHAPITRE 4 — PROJET SIKBEAUTY — MIGRATION ET INDUSTRIALISATION

4.1 Présentation et contexte

SikBeauty est une marketplace e-commerce spécialisée dans la vente de cosmétiques coréens (K-Beauty). La plateforme propose un catalogue de produits organisés par catégories et marques, un système de panier d'achat, une gestion des commandes et un espace client complet. Au moment de notre intervention, l'application était développée avec React (Create React App), une Single Page Application (SPA) standard, et souffrait de deux problèmes majeurs affectant directement son potentiel commercial.

Le premier problème était d'ordre **technique** : les temps de chargement initiaux étaient élevés (FCP > 2.5s) en raison du bundle JavaScript important à télécharger et à exécuter côté client avant tout rendu. Le second était d'ordre **commercial** : les moteurs de recherche (Google, Bing) ne pouvaient pas indexer correctement les fiches produits, car leur contenu était généré dynamiquement par JavaScript après le chargement initial. Le score SEO Lighthouse de la page d'accueil était de 68/100, et celui des fiches produits était encore plus faible.

4.2 Migration React vers Next.js et optimisation SSR/SEO

Face à ces enjeux, la décision a été prise de migrer la partie commerciale critique de l'application (page d'accueil, catalogue, fiches produits) vers **Next.js avec l'App Router**. Cette architecture permet le Server-Side Rendering (SSR) pour les contenus dynamiques et le Static Site Generation (SSG) pour les contenus peu changeants, offrant le meilleur de deux mondes : contenu pré-rendu accessible aux robots de recherche et expérience utilisateur réactive.

J'ai débuté par une phase d'**audit complet** des composants React existants, identifiant les trois catégories de problèmes à résoudre lors de la migration :

- Les dépendances aux APIs du navigateur (`window`, `localStorage`) incompatibles avec l'exécution côté serveur.
- Les hooks React utilisant `useEffect` avec des effets de bord immédiats, incompatibles avec le rendu serveur.
- Les bibliothèques tierces sans support SSR (notamment les bibliothèques de graphiques et de carrousels).

Pour chaque page migrée, j'ai appliqué la même démarche systématique : transformer les composants statiques en **Server Components** (pour les données ne nécessitant pas d'interactivité), annoter les composants interactifs avec la directive `'use client'`, et gérer les imports de bibliothèques non-SSR via `next/dynamic` avec l'option `ssr: false`.

Grâce à mon expérience préalable avec React et Next.js, j'ai également assuré un rôle de **mentor technique** auprès des autres développeurs de l'équipe SikBeauty, qui découvraient les spécificités de l'App Router pour la première fois. J'ai organisé des sessions de pair-programming ciblées et rédigé des guides internes documentant les patterns à suivre et les anti-patterns à éviter.

4.2.1 Optimisation SEO technique

En tant que responsable SEO du projet, j'ai implémenté une stratégie complète d'optimisation du référencement naturel, tirant parti des capacités natives de Next.js. La génération des métadonnées est entièrement dynamique, calculée côté serveur à partir des données produit récupérées en base.

TECHNIQUE SEO	AVANT (REACT SPA)	APRÈS (NEXT.JS SSR)	IMPACT
Time To First Byte (TTFB)	1.2s	0.25s	↓ 79% — Réponse quasi instantanée.
First Contentful Paint (FCP)	2.8s	0.8s	↓ 71% — Affichage immédiat du contenu.
Largest Contentful Paint (LCP)	4.1s	1.4s	↓ 66% — Core Web Vital dans la zone verte.
Indexation fiches produits	Partielle (JS requis)	100% automatique	Toutes les fiches indexées par Google.
Score Performance Lighthouse	52/100	91/100	+39 points — Amélioration majeure.
Score SEO Lighthouse	68/100	98/100	+30 points — SEO quasi parfait.
Meta description dynamique	Absente	Générée par produit	CTR amélioré dans les SERP.
Open Graph + Twitter Cards	Statiques	Dynamiques par page	Partage sur réseaux sociaux optimisé.

Tableau 3 : Comparatif React SPA vs Next.js SSR — Métriques de performance et SEO

OPTIMISATION SEO	DESCRIPTION
title dynamique	Généré avec generateMetadata() par page — format : "Produit | SikBeauty".
meta description	Extrait des 160 premiers caractères de la description produit.
canonical URLs	Attribut canonical sur chaque page pour éviter le contenu dupliqué.
Open Graph (og:*)	og:title, og:description, og:image, og:type — pour le partage réseaux sociaux.
Twitter Cards	twitter:card, twitter:title, twitter:image — pour l'aperçu sur Twitter/X.
sitemap.xml	Généré dynamiquement via sitemap.ts — toutes les fiches produits incluses.
robots.txt	Fichier configuré pour autoriser l'indexation des pages publiques.
next/image	Lazy loading automatique, formats WebP/AVIF, dimensionnement responsive.
Structured Data (JSON-LD)	Markup Product pour une éligibilité aux Rich Results Google.

Tableau 4 : Métriques SEO Lighthouse avant et après migration

4.3 DevOps, Docker et CI/CD GitHub Actions

Une fois la migration technique réalisée, j'ai conçu et mis en place l'intégralité de l'infrastructure de déploiement et de livraison continue du projet SikBeauty.

4.3.1 Conteneurisation Docker multi-stage

J'ai rédigé un **Dockerfile multi-stage** optimisé pour Next.js en mode standalone. Cette approche réduit drastiquement la taille de l'image de production en séparant la phase de build (qui inclut toutes les dépendances de développement, les outils de compilation TypeScript, etc.) de la phase de production (qui ne contient que le serveur Node.js standalone et les assets statiques).

```
# Étape 1 : Builder (inclut devDependencies + compilation)
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Étape 2 : Production (image minimale ~140Mo vs ~950Mo)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

Le fichier `docker-compose.yml` orchestre les services de l'environnement de développement : l'application Next.js, un conteneur PostgreSQL (pour le backend), et un service Redis. Les volumes Docker garantissent la persistance des données entre les redémarrages et le hot-reload du code pendant le développement.

4.3.2 Pipeline CI/CD GitHub Actions

J'ai rédigé le workflow GitHub Actions (fichier `.github/workflows/deploy.yml`) assurant un pipeline de livraison continue fiable. Le pipeline est déclenché sur chaque push vers les branches `main` et `dev` et se compose des étapes suivantes :

1. **Installation des dépendances** (`npm ci`) avec mise en cache des `node_modules`.
2. **Vérification des types TypeScript** (`npx tsc --noEmit`) — bloquant en cas d'erreur.
3. **Lint du code** (ESLint) — bloquant en cas de violations de règles critiques.
4. **Build de production** (`npm run build`) — vérification de la compilation.
5. **Connexion SSH au VPS Hostinger** via la clé stockée dans les GitHub Secrets.
6. **Déploiement via Docker Compose** : pull de la dernière image, recréation du conteneur, migration de la base de données.

4.4 Déploiement sur VPS Hostinger

Le déploiement en production s'est fait sur un VPS Ubuntu hébergé chez **Hostinger**. La configuration du serveur a nécessité plusieurs étapes : installation de Docker Engine et Docker Compose, configuration du firewall (UFW) pour n'exposer que les ports 80, 443 et 22, installation et configuration de **Nginx** comme reverse proxy pour acheminer le trafic vers le conteneur applicatif, et mise en place d'un certificat SSL automatique via **Let's Encrypt** (Certbot). Le nom de domaine a été configuré dans l'interface Hostinger en pointant les enregistrements DNS A vers l'adresse IP publique du VPS.

4.5 Contributions Backend

En complément de mes contributions principales, j'ai également apporté mon aide à l'équipe backend de SikBeauty en corrigeant plusieurs problèmes identifiés lors des revues de code et des tests : correction d'une fuite de connexions Prisma en environnement de test, résolution d'une incohérence de validation des données d'entrée sur l'API de gestion des commandes, et optimisation d'une requête Prisma générant des appels N+1 sur la liste des produits.

CHAPITRE 5 — PROJET ALIGNO — CONCEPTION ET DÉVELOPPEMENT FROM SCRATCH

5.1 Présentation d'Aligno

Aligno est une application web Full-Stack de gestion de la productivité et du mode de vie, conçue pour harmoniser le travail, la gestion du temps et les finances personnelles sous une interface unique et cohérente. L'application intègre quatre modules fonctionnels principaux :

- **Kanban (Project Management)** : tableaux de bord collaboratifs avec colonnes, cartes, sous-tâches, labels et assignations, synchronisés en temps réel via Socket.IO.
- **Time Management** : système de blueprints de journée (templates de blocs de temps) permettant d'activer des plannings quotidiens personnalisés.
- **Wallet** : gestion multi-comptes bancaires, suivi des revenus et dépenses, gestion des abonnements récurrents, budgets mensuels et analyses graphiques.
- **Dashboard** : vue synthétique agrégeant les indicateurs clés de productivité et de santé financière.

La stack technique d'Aligno reflète une ambition de modernité et de performance : **Nuxt 4** comme framework Full-Stack, **Vue 3 avec Composition API**, **TypeScript** strict, **Tailwind CSS v4** avec le design system Midnight Obsidian, **Nuxt UI** pour les composants, **Pinia** pour la gestion d'état, **Prisma ORM** avec **PostgreSQL 17**, **Redis** pour le cache et les queues **BullMQ**, et **Socket.IO** pour la collaboration en temps réel.

5.2 Initialisation du projet et configuration

J'ai pris en charge l'initialisation complète du projet Aligno. Cette phase fondatrice, souvent sous-estimée, conditionne la qualité et la maintenabilité de l'ensemble du développement. Elle comprenait :

- La création du projet Nuxt 4 et la configuration du fichier `nuxt.config.ts` (modules, runtimeConfig, Tailwind, sécurité).
- La mise en place de la structure de répertoires suivant le pattern layered architecture.
- La configuration de **Prisma ORM** avec les modèles de base de données et le premier schéma.

- La rédaction du `docker-compose.yml` orchestrant les quatre services : Nuxt App, PostgreSQL 17, Redis 7 Alpine, et pgAdmin 4 pour l'administration de la base.
- La configuration des hooks Git via **Husky** : hook `pre-push` exécutant automatiquement la suite de tests Vitest dans le conteneur Docker avant chaque push.
- La configuration du pipeline CI GitHub Actions avec les actions PostgreSQL intégrées.

VARIABLE D'ENVIRONNEMENT	REQUIS	DESCRIPTION
DATABASE_URL	✅	Chaîne de connexion PostgreSQL (prisma+pg adapter).
JWT_SECRET	✅	Clé secrète pour signer les access tokens (15 min).
JWT_REFRESH_SECRET	✅	Clé secrète pour signer les refresh tokens (30 jours).
RESEND_API_KEY	✅	Clé API Resend pour les e-mails transactionnels.
APP_URL	❌	URL de base de l'application (défaut : http://localhost:4000).
NUXT_PORT	❌	Port d'écoute du serveur Nitro (défaut : 4000).
LOG_LEVEL	❌	Niveau de log Pino (défaut : info).
SENTRY_DSN	❌	DSN Sentry pour le tracking des exceptions en production.

Tableau 5 : Variables d'environnement du projet Aligno

5.3 Développement Backend du module Authentification

J'étais responsable de la conception et de l'implémentation complète du **module d'authentification** d'Aligno, qui constitue la fondation sécuritaire de toute l'application. Ce module, d'une complexité technique élevée, implémente un système d'authentification dual-token avec renouvellement silencieux, validation d'e-mail, et récupération de mot de passe.

5.3.1 Architecture en 6 couches

Chaque endpoint de l'API d'authentification traverse strictement les 6 couches de l'architecture :

1. **Route Handler** (`server/api/auth/*.ts`) : point d'entrée HTTP. Définit la méthode HTTP, le path, et les spécifications OpenAPI/Swagger. Délègue immédiatement au Controller.
2. **Controller** (`server/controller/auth.controller.ts`) : parse le corps de la requête, déclenche la validation Zod, gère les cookies HttpOnly, formate l'enveloppe de réponse JSON standardisée (`{ success, data }` ou `{ success, message, errors }`).
3. **Service** (`server/service/auth.service.ts`) : orchestre la logique métier. Ne connaît pas la requête HTTP. Coordonne le hachage des mots de passe (bcrypt), la génération et vérification des tokens JWT, et l'envoi d'e-mails (Resend).
4. **Repository** (`server/repository/user.repository.ts`) : implémentation concrète de l'interface DAO. Contient toutes les requêtes Prisma vers PostgreSQL.
5. **DAO** (`server/dao/user.dao.ts`) : interface TypeScript définissant le contrat d'accès aux données. Permet de substituer facilement l'implémentation (ex : passer de PostgreSQL à MongoDB sans modifier les couches supérieures).
6. **DTO / Mapper** (`server/dto/auth.dto.ts`, `server/mapper/user.mapper.ts`) : DTOs TypeScript assurant la sécurité de type entre les couches. Le Mapper garantit que les champs sensibles (mot de passe hashé, tokens de reset) ne sont jamais exposés dans les réponses API.

5.3.2 Endpoints développés

ENDPOINT	MÉTHODE	AUTH	DESCRIPTION
`/api/auth/register`	POST	Guest Only	Inscription, upload avatar WebP (Sharp), envoi e-mail de vérification Resend.
`/api/auth/login`	POST	Guest Only	Authentification bcrypt, émission du couple JWT (access 15min + refresh 30j cookie HttpOnly).
`/api/auth/me`	GET	Bearer JWT	Récupération du profil utilisateur (champs sensibles strippés par le Mapper).
`/api/auth/logout`	POST	Bearer JWT	Révocation du refresh token en DB, suppression du cookie HttpOnly.
`/api/auth/email/validation`	POST	Guest Only	Vérification du jeton e-mail (SHA-256 comparé avec le hash en DB).
`/api/auth/forgot-password`	POST	Guest Only	Génération d'un token de reset (1h), envoi e-mail Resend. Réponse identique que l'e-mail existe ou non (anti-énumération).
`/api/auth/reset-password`	POST	Guest Only	Vérification du token de reset, hachage du nouveau mot de passe, invalidation du token.
`/api/auth/profile`	PUT	Bearer JWT	Mise à jour des informations de profil et/ou de l'avatar (compression Sharp, suppression de l'ancienne image).

Tableau 6 : Registre complet des endpoints API du module Authentification

5.3.3 Sécurité et validations

CONTRÔLE DE SÉCURITÉ	IMPLÉMENTATION TECHNIQUE
Hachage des mots de passe	Bcrypt avec 12 rounds de sel. Le mot de passe en clair n'est jamais stocké ni loggé.
Access Token JWT	Durée de vie : 15 minutes. Signé avec JWT_SECRET. Transmis en header Authorization: Bearer.
Refresh Token JWT	Durée de vie : 30 jours. Signé avec JWT_REFRESH_SECRET. Stocké en cookie HttpOnly, Secure, SameSite=Strict. Seul son hash SHA-256 est persisté en base.
Renouvellement silencieux	Le middleware Auth intercepte les access tokens expirés et tente un renouvellement via le cookie refresh. Le nouveau token est retourné dans X-Access-Token.
Tokens de vérification/reset	Générés avec crypto.randomBytes(32). Seul leur hash SHA-256 est stocké en base.
Anti-énumération utilisateurs	forgot-password retourne le même message 200 que l'e-mail existe ou non.
Validation des entrées (Zod)	Tous les payloads sont validés par des schémas Zod stricts. Retour 422 avec erreurs par champ.
Stripping des champs sensibles	UserMapper.toResponseDto() supprime passwordHash, refreshToken, emailVerifyToken des réponses.
Traitement des images	Sharp redimensionne et compresse en WebP 400×400px. L'ancienne image est supprimée du disque.

Tableau 8 : Contrôles de sécurité implémentés dans le module Auth d'Aligno

5.4 Tests unitaires et d'intégration avec Vitest

J'ai développé l'ensemble de la suite de tests du module Authentification, couvrant 35 scénarios distincts répartis en 9 fichiers de tests. La stratégie de test adopte une approche d'**intégration complète** : les tests ne mockent pas la base de données mais utilisent un serveur Nuxt réel (`@nuxt/test-utils/e2e`) connecté à une instance PostgreSQL de test, garantissant que chaque test valide le comportement de bout-en-bout, de la requête HTTP jusqu'à la persistance en base.

La configuration Vitest impose l'exécution **séquentielle des fichiers** (`fileParallelism: false`) pour éviter les race conditions sur la base de données partagée. Chaque fichier de test nettoie la base de données dans un `beforeEach()` via la fonction `clearDatabase()`, garantissant l'isolation et le déterminisme des tests.

FICHIER DE TEST	NB TESTS	SCÉNARIOS COUVERTS
register.test.ts	8	Inscription réussie, e-mail dupliqué, téléphone dupliqué, prénom manquant, e-mail invalide, mot de passe trop court, confirmPassword ne correspond pas, champs obligatoires manquants.
login.test.ts	5	Connexion réussie + cookie refresh, utilisateur inexistant (401), mauvais mot de passe (401), e-mail non vérifié (403), e-mail manquant (422).
me.test.ts	3	Profil avec token valide (200), token manquant (401), token expiré (401).
logout.test.ts	2	Déconnexion réussie + révocation cookie, non autorisé (401).
validation.test.ts	3	Token de vérification valide, token invalide (400), token expiré (400).
forgot-password.test.ts	3	E-mail existant (200 — réponse générique), e-mail inexistant (200 — anti-énumération), e-mail manquant (422).
reset-password.test.ts	4	Reset valide, token invalide (400), token expiré (400), mot de passe manquant (422).
auth-middleware.test.ts	5	Access token valide, aucun credential (401), renouvellement silencieux avec X-Access-Token, cookie refresh invalide (401), hash refresh token ne correspond pas en DB (401).
guest-middleware.test.ts	2	Utilisateur authentifié bloqué sur /login (403), bloqué sur /register (403).

Tableau 9 : Couverture des tests Vitest — Module Authentification (35 tests)

5.5 Planification Frontend sur Jira

Une fois le backend d'authentification terminé et testé, j'ai préparé la **planification complète du développement frontend** sur Jira, avant que les autres développeurs ne commencent à coder les interfaces. Cette phase de planification est cruciale pour éviter les doublons de travail, les conflits Git et les incohérences d'architecture.

J'ai créé une Epic "Module Auth — Frontend" regroupant toutes les User Stories du module. Chaque User Story a été découpée en tâches précises avec des critères d'acceptation détaillés,

puis distribuée aux membres de l'équipe en fonction de leur affinité avec chaque fonctionnalité. Les tâches ont été priorisées selon les dépendances fonctionnelles : le store Pinia et la couche API devaient être implémentés en premier, car toutes les pages en dépendaient.

5.6 Développement Frontend — Module Auth (Nuxt 4)

J'ai développé toutes les interfaces du module d'authentification avec **Nuxt 4**, **Vue 3 (Composition API)**, **TypeScript**, **Tailwind CSS v4**, **Nuxt UI** et **Pinia**, en respectant le design system Midnight Obsidian défini en Sprint 0.

5.6.1 Store Pinia et composables

L'état d'authentification est géré par un store Pinia persisté (`pinia-plugin-persistedstate`). Contrairement au refresh token (stocké en cookie HttpOnly côté serveur), l'access token est conservé **uniquement en mémoire** (non persisté), pour des raisons de sécurité. Lors d'un rechargement de page, le composable `useAuth` effectue automatiquement une requête vers `/api/auth/me`, qui utilise le cookie refresh HttpOnly pour revalider la session et obtenir un nouveau access token.

5.6.2 Pages développées

Les pages du module Auth suivent toutes le même layout `auth.vue`, qui applique le thème Midnight Obsidian (fond sombre avec dégradé, effets glassmorphism). Chaque page implémente la validation côté client avant tout appel API, en utilisant des règles identiques aux schémas Zod côté serveur pour une expérience cohérente :

- **Register** : formulaire multi-champs avec upload d'avatar (prévisualisation en temps réel), indicateur de force du mot de passe (composant `PasswordStrength.vue`), sérialisation en `FormData` pour l'upload multipart.
- **Login** : gestion de la redirection post-connexion (`?redirect=`), affichage spécifique si l'e-mail n'est pas encore vérifié (code 403).
- **Forgot Password** : timer de 60 secondes pour éviter le spam de requêtes, avec feedback visuel du compte à rebours.
- **Reset Password** : vérification immédiate de la présence du token dans l'URL au montage du composant, avec affichage d'un écran d'erreur adapté si le token est absent ou invalide.
- **Verify Email** : validation automatique au montage si un token est présent en query param (`?token=`), avec un input de fallback pour saisie manuelle du code.
- **Profile** : affichage des données utilisateur, formulaire de mise à jour partielle des champs, upload d'un nouvel avatar avec compression et compression d'avertissement.

5.7 Phase de finalisation du projet

J'ai été responsable de la **phase de finalisation** du projet Aligno, précédant la livraison de la version de production. Cette phase, souvent sous-estimée, est pourtant déterminante pour la qualité perçue de l'application et sa viabilité à long terme.

- **SEO et métadonnées** : Implémentation de useHead() sur toutes les pages de l'application avec des titres, meta descriptions, Open Graph et URLs canoniques pertinents.
- **Optimisation des performances** : Audit des bundles JavaScript avec `nuxt analyze`, lazy loading des composants non critiques, optimisation des images avec `@nuxt/image`.
- **Accessibilité (WCAG AA)** : Audit de l'arborescence ARIA, contraste des couleurs, navigation clavier, labels des formulaires, attributs alt sur toutes les images.
- **Dark Mode / Light Mode** : Intégration du mécanisme de thème système de Nuxt UI, persistance du choix dans le store Pinia.
- **Système de notifications** : Intégration de **Notivue** comme système de toast global, avec des notifiations de succès, d'erreur et d'information cohérentes dans toute l'application.
- **Fusion des branches et livraison** : Gestion des Pull Requests finales, résolution des conflits Git, merge dans `main`, validation du pipeline CI complet, livraison du build de production.

CHAPITRE 6 — PRÉSENTATION DES TECHNOLOGIES

TECHNOLOGIE	VERSION	RÔLE	AVANTAGES / UTILISATION CONCRÈTE
Nuxt 4	4.x	Framework Full-Stack	SSR, file-based routing, server Nitro intégré. Utilisé comme unique framework pour le frontend ET le backend d'Aligno.
Vue 3	3.5+	Framework UI	Composition API, réactivité fine, slots scopés. Tous les composants d'interface d'Aligno sont en Vue 3.
TypeScript	5.x	Typage statique	Détection d'erreurs à la compilation, contrats d'interface entre les couches. Strict mode activé.
Tailwind CSS v4	4.x	Framework CSS	Utility-first, Vite plugin natif, sans conflict avec Nuxt UI. Design system Midnight Obsidian défini avec des variables CSS.
Nuxt UI	3.x	Composants UI	Bibliothèque de composants (Button, Input, Modal, Table…) construite sur Tailwind v4. Système de thèmes intégré.
Pinia	2.x	State Management	Store réactif avec support TypeScript natif. Stores : useAuthStore, useWalletStore, useKanbanStore…
Pinia Persisted	3.x	Persistance	Persistance sélective des stores dans localStorage (user, préférences de thème). Access token non persisté.
ApexCharts	3.x	Graphiques	Graphiques analytiques du module Wallet (revenus/dépenses par mois, répartition par catégorie).
VueUse	12.x	Composables utils	useLocalStorage, useEventListener, useMediaQuery… Réutilisation de patterns courants.

TECHNOLOGIE	VERSION	RÔLE	AVANTAGES / UTILISATION CONCRÈTE
Next.js	14.x	Framework SSR	App Router avec SSR/SSG pour SikBeauty. generateMetadata pour le SEO dynamique.

Tableau 10 : Fiches technologiques — Stack Frontend

TECHNOLOGIE	VERSION	RÔLE	AVANTAGES / UTILISATION CONCRÈTE
Prisma ORM	5.x	Accès BDD	Schéma typé, migrations auto, requêtes type-safe. Adapter pg pour pool de connexions PostgreSQL.
PostgreSQL	17	Base de données	ACID, relations, UUID/CUID. Tables : users, projects, kanban_columns, cards, transactions, bank_accounts…
Zod	3.x	Validation	Schémas de validation runtime avec inférence TypeScript. Chaque endpoint API dispose de son schéma Zod dédié.
JWT (jose)	5.x	Tokens	Génération et vérification des access (15min) et refresh (30j) tokens. jose pour un support ESM natif.
bcrypt	5.x	Hachage MDP	12 rounds de sel. Comparaison à temps constant pour résister aux attaques timing.
Resend	3.x	E-mails	API d'envoi d'e-mails transactionnels. Templates en Vue Email (VerificationEmail, PasswordResetEmail).
Socket.IO	4.x	Temps réel	Mise à jour en temps réel du Kanban (déplacement de cartes, notifications). Connecté avec Redis Adapter.
Redis	7.x	Cache + Queue	Broker pour Socket.IO Adapter (sync multi-instances) et backend de BullMQ (queues d'e-mails différés).
BullMQ	5.x	File de messages	Jobs de rappels de dates d'échéance (due date reminders) pour les cartes Kanban.
Pino	9.x	Logging	Logger JSON structuré haute performance. pino-pretty pour le dev, JSON brut pour la production/Sentry.
Sentry	8.x	Monitoring	Capture des exceptions runtime (désactivé en NODE_ENV=test). DSN configuré en variable d'environnement.

Tableau 11 : Fiches technologiques — Stack Backend et Temps Réel

TECHNOLOGIE	RÔLE	UTILISATION CONCRÈTE
Docker	Conteneurisation	Dockerfile multi-stage pour Next.js (SikBeauty) et Nuxt 4 (Aligno). Image de prod ~140Mo.
Docker Compose	Orchestration	Services : app, postgres_db, redis, pgadmin. Health checks et depends_on pour l'ordre de démarrage.
GitHub Actions	CI/CD	Pipelines : install, generate, type-check, migrate, test, (deploy SSH). Déclenchés sur PR vers main/dev.
Hostinger VPS	Hébergement	Ubuntu Server. Nginx reverse proxy + Let's Encrypt SSL + Docker Compose en prod.
Git / GitHub	Versioning	Git Flow : main, dev, feature/*, fix/*. PRs obligatoires, revues de code systématiques.
Vitest	Tests	35 tests d'intégration Auth (Aligno). Sequential mode pour éviter les race conditions PostgreSQL.
Husky	Git Hooks	Hook pre-push : exécute vitest run dans le conteneur Docker avant chaque push.
Nuxt Security	Sécurité	Headers CSP, X-Frame-Options, X-Content-Type-Options, CORS configurés automatiquement.

Tableau 12 : Fiches technologiques — DevOps, Qualité et Sécurité

CHAPITRE 7 — DIFFICULTÉS RENCONTRÉES — ÉTUDES DE CAS

Cette section présente les difficultés techniques et organisationnelles les plus significatives rencontrées durant le stage, sous forme d'études de cas structurées. Chaque cas est analysé rigoureusement afin d'en tirer des enseignements durables.

▶ Cas 1 : Erreur de compilation vue-tsc dans le pipeline CI/CD

Contexte : Configuration du pipeline GitHub Actions pour le projet Aligno. Node.js v24 était utilisé dans l'environnement CI.

Problème : La commande `npx vue-tsc --noEmit` échouait avec l'erreur `ERR_PACKAGE_PATH_NOT_EXPORTED` : le sous-chemin `./lib/tsc` n'était pas exporté par le package TypeScript installé. Le pipeline CI était bloquant.

Analyse : L'exécuteur `npx` installait une version globale de `vue-tsc` (3.3.7) qui tentait d'importer `./lib/tsc` depuis TypeScript. Cette exportation n'est plus disponible dans TypeScript ≥ 5.0 via le nouveau champ `exports` du package.json. La version globale de vue-tsc était incompatible avec la version locale de TypeScript.

Solution : Remplacement de `npx vue-tsc --noEmit` par `./node_modules/.bin/vue-tsc --noEmit` dans le workflow CI. Ajout de `vue-tsc` dans les `devDependencies` du `package.json` pour garantir sa présence après `npm ci`.

Résultat : Le pipeline CI passe en totalité. La vérification des types TypeScript est désormais stable et reproductible dans tous les environnements (local et CI).

Enseignement : Toujours préférer les binaires locaux (via `node_modules/.bin/`) aux outils globaux installés via `npx` dans les pipelines CI, afin d'assurer la cohérence des versions entre développement et intégration continue.

▶ Cas 2 : Intégration de Resend dans les tests Vitest

Contexte : Exécution de la suite de tests d'authentification en environnement de CI GitHub Actions, avec une clé API Resend de test factice.

Problème : Les tests de l'endpoint `/api/auth/register` et `/api/auth/forgot-password` échouaient avec une erreur 401 de l'API Resend, car la clé API configurée dans les secrets CI était invalide. La fonctionnalité d'envoi d'e-mail est obligatoire dans le flux d'inscription.

Analyse : En environnement de test, les e-mails ne doivent pas être réellement envoyés. Mock complet du service Resend aurait été une option, mais cela aurait réduit la valeur des tests d'intégration. La solution devait être transparente pour le reste du code.

Solution : Implémentation d'une logique `shouldSkip()` dans le EmailService : si `NODE_ENV === 'test'` OU si le domaine de l'e-mail destinataire est `example.com`, l'appel Resend est bypassé et un message est loggué. Les tokens de vérification sont quand même générés et persistés, permettant aux tests de les récupérer directement en base.

Résultat : Les 35 tests passent en CI sans aucun appel réel à l'API Resend. La logique d'authentification est testée de bout-en-bout sans dépendance externe. Le comportement en production reste inchangé.

Enseignement : La gestion des dépendances externes (APIs tierces, services e-mail) dans les tests d'intégration nécessite des mécanismes de contournement conditionnels. Il vaut mieux implémenter un bypass basé sur l'environnement que de mocker entièrement un service, car cela préserve la valeur des tests d'intégration.

▶ Cas 3 : Migration React vers Next.js — Incompatibilités SSR

Contexte : Migration des pages de la marketplace SikBeauty de React (Create React App) vers Next.js App Router.

Problème : Plusieurs composants React utilisaient directement des APIs du navigateur (`window.scrollY`, `localStorage.getItem`, `document.querySelector`) dans des hooks `useEffect` ou dans le corps de composants fonctionnels. Ces accès provoquaient des erreurs de type `ReferenceError: window is not defined` lors du rendu côté serveur.

Analyse : Next.js exécute les Server Components et le rendu initial des Client Components côté serveur (Node.js), où `window`, `document` et `localStorage` n'existent pas. Ces accès doivent être soit protégés par une condition `typeof window !== 'undefined'`, soit déplacés dans des hooks `useEffect` qui s'exécutent uniquement côté client.

Solution : Audit systématique de tous les composants avec un script de recherche grep identifiant les occurrences de `window`, `document` et `localStorage`. Refactorisation : (1) extraction dans des `useEffect` avec tableau de dépendances vide, (2) utilisation de `next/dynamic` avec `ssr: false` pour les composants non-SSR-compatibles, (3) remplacement de `localStorage` par `useCookie` de Nuxt (accessible côté serveur).

Résultat : Tous les composants migrés s'exécutent sans erreur en SSR. Le TTFB passe de 1.2s à 0.25s. L'indexation Google des fiches produits passe de partielle à 100%.

Enseignement : La migration vers SSR requiert une revue architecturale préalable rigoureuse. Les patterns React classiques (accès DOM dans le corps des composants) ne sont pas directement transposables. Une migration réussie impose un audit complet avant de commencer.

CHAPITRE 8 — BILAN DES COMPÉTENCES ACQUISES

COMPÉTENCE TECHNIQUE	NIVEAU AVANT LE STAGE	NIVEAU APRÈS LE STAGE	PREUVE CONCRÈTE
Architecture logicielle (couches)	Débutant	Avancé	Conception et implémentation de l'architecture 6 couches d'Aligno.
Développement Backend Node.js	Intermédiaire	Expert	Module Auth complet avec sécurité avancée JWT, bcrypt, Zod.
Tests d'intégration (Vitest)	Débutant	Avancé	35 tests d'intégration couvrant 9 fichiers et 100% des endpoints Auth.
DevOps & CI/CD	Débutant	Autonome	Pipelines GitHub Actions, Dockerfiles multi-stage, déploiement VPS.
SSR / Next.js App Router	Intermédiaire	Expert	Migration SikBeauty : SEO 98/100 Lighthouse, TTFB 0.25s.
SEO technique	Notions	Avancé	Métadonnées dynamiques, sitemap, Open Graph, robots.txt, Structured Data.
Sécurité Web	Notions	Avancé	Dual-token JWT, HttpOnly cookies, anti-énumération, hachage SHA-256.
TypeScript avancé	Intermédiaire	Expert	DTOs génériques, interfaces DAO, type guards, strict mode complet.
PostgreSQL + Prisma ORM	Intermédiaire	Avancé	Schéma complet, migrations, requêtes complexes avec relations include.

COMPÉTENCE TECHNIQUE	NIVEAU AVANT LE STAGE	NIVEAU APRÈS LE STAGE	PREUVE CONCRÈTE
Accessibilité (WCAG)	Notions	Intermédiaire	Audit ARIA, contraste, navigation clavier appliqués en phase finale.

Tableau 13 : Bilan comparatif des compétences techniques — Avant et Après le stage

COMPÉTENCE TRANSVERSALE	NIVEAU AVANT	NIVEAU APRÈS	SITUATION RÉVÉLATRICE
Leadership technique	Aucun	Confirmé	Direction de l'équipe de 5 développeurs sur 2 projets simultanés.
Scrum Master	Théorique	Opérationnel	Animation de 8 sprints, planification, résolution des blocages.
Revue de code	Notions	Avancé	Review de 100% des PR sur les deux projets, refactorisation guidée.
Communication technique	Intermédiaire	Expert	Documentation, User Stories, sessions de pair-programming, Daily Meetings.
Résolution de problèmes	Intermédiaire	Expert	Résolution des 3 cas d'étude présentés dans le chapitre précédent.
Gestion du temps & priorisation	Intermédiaire	Avancé	Gestion simultanée de 2 projets avec planification Jira rigoureuse.
Mentorat	Aucun	Confirmé	Accompagnement des développeurs lors de la migration Next.js.

Tableau 14 : Bilan des compétences transversales et managériales — Avant et Après le stage

CHAPITRE 9 — RÉSULTATS OBTENUS

LIVRABLE / OBJECTIF	INDICATEUR DE SUCCÈS	RÉSULTAT OBTENU	STATUT
Migration SikBeauty (Next.js)	Score SEO Lighthouse ≥ 90	98/100 Lighthouse SEO	✅ Dépassé
Performance SikBeauty	FCP < 1.5s	FCP = 0.8s	✅ Dépassé
Infrastructure CI/CD	Pipeline fonctionnel sur PR	Pipeline en 5 étapes complet	✅ Atteint
Déploiement VPS	Application accessible en prod	App + SSL + Domaine configurés	✅ Atteint
Module Auth Backend (Aligno)	Tous les endpoints implémentés	8 endpoints + 2 middlewares	✅ Complet
Tests d'intégration Auth	Couverture ≥ 30 tests	35 tests, 100% passing	✅ Dépassé
Module Auth Frontend	Toutes les pages Auth livrées	6 pages + composants réutilisables	✅ Complet
Coordination Scrum Master	4 sprints livrés dans les délais	4 sprints, 0 retard de livraison	✅ Atteint
Phase finale Aligno	Dark Mode + SEO + Accessibilité	Tous les critères validés	✅ Complet
Documentation technique	Docs à jour (READMEs + Swagger)	Swagger + 11 fichiers docs/	✅ Atteint

Tableau 15 : Récapitulatif des indicateurs clés et résultats du stage

CONCLUSION GÉNÉRALE

Ce stage de fin de formation au sein de Landmark Agency a représenté bien plus qu'une simple validation des acquis théoriques. Il s'est révélé être une véritable immersion dans la réalité des défis d'un développement professionnel : gestion de la complexité technique, coordination humaine, respect des délais, prise de décision sous contrainte et responsabilité vis-à-vis d'une équipe et d'un client.

Sur le plan **technique**, les deux mois de stage m'ont permis de consolider et d'approfondir un spectre de compétences très large. La migration de SikBeauty m'a confronté aux réalités de la modernisation d'un système existant, avec toutes ses contraintes de compatibilité et ses exigences de performance. Le développement d'Aligno m'a offert l'opportunité, rare en stage, de concevoir et d'implémenter une architecture logicielle complète de zéro, en faisant des choix techniques réfléchis et en les assumant tout au long du projet. La mise en place du pipeline DevOps et la gestion de l'infrastructure de production m'ont donné une vision globale du cycle de vie d'une application web moderne, de la première ligne de code jusqu'au déploiement en production.

Sur le plan **méthodologique et humain**, le rôle de Scrum Master a été la dimension la plus formatrice de ce stage. Coordonner une équipe de cinq personnes, animer des cérémonies Scrum, résoudre des conflits et maintenir une cadence de livraison régulière ont développé des compétences de leadership que l'apprentissage individuel ne peut pas reproduire. J'ai appris que la qualité d'une livraison logicielle dépend autant de la qualité de la communication et de l'organisation que de la qualité du code lui-même.

L'expérience chez Landmark Agency m'a également confirmé dans mon orientation professionnelle. Le double rôle de Lead Developer et de Scrum Master correspond parfaitement à mes ambitions : je souhaite évoluer vers un profil d'ingénieur Full-Stack spécialisé en architecture logicielle et DevOps, capable à la fois de concevoir des systèmes complexes et de conduire une équipe technique vers ses objectifs.

Les perspectives ouvertes par ce stage sont nombreuses. Les projets SikBeauty et Aligno continueront d'évoluer et d'accueillir de nouvelles fonctionnalités. La qualité de l'architecture mise en place sur Aligno garantit une scalabilité sereine de l'application. À titre personnel, ce stage constitue un tremplin solide vers une carrière d'ingénieur en développement logiciel, avec une feuille de route claire : approfondir mes connaissances en architecture distribuée, en orchestration Kubernetes et en pratiques de sécurité avancées.

CHAPITRE — — ANNEXES

Annexe A : Structure des répertoires du projet Aligno

```
Aligno/
├── app/                        # Frontend Nuxt 4 (Vue 3)
│   ├── pages/                  # Routes (login, register, dashboard...)
│   ├── components/             # Composants Vue réutilisables
│   ├── stores/                 # Stores Pinia (auth, wallet, kanban)
│   ├── composables/            # Composables (useAuth, useWallet...)
│   ├── layouts/                # Layouts (auth, default)
│   └── types/                  # Interfaces TypeScript partagées
├── server/                     # Backend Nitro (API REST)
│   ├── api/                    # Route Handlers (GET, POST, PUT...)
│   ├── controller/             # Controllers (validation, réponses)
│   ├── service/                # Services (logique métier)
│   ├── repository/             # Repositories (Prisma queries)
│   ├── dao/                    # Interfaces DAO (contrats)
│   ├── dto/                    # Data Transfer Objects
│   ├── mapper/                 # Mappers (entity → DTO)
│   ├── middleware/             # Auth + Guest middleware
│   ├── validations/            # Schémas Zod par endpoint
│   ├── emails/                 # Templates Vue Email (Resend)
│   └── utils/                  # Prisma, Logger, Crypto, Sentry
├── prisma/                     # Schémas Prisma + migrations
│   ├── models/                 # user.prisma, project.prisma...
│   └── migrations/             # Historique des migrations SQL
├── server/test/                # Suite de tests Vitest
│   └── auth/                   # 9 fichiers × 35 tests
├── .github/workflows/          # CI/CD GitHub Actions
├── docker-compose.yml          # Orchestration locale
├── Dockerfile                  # Image Docker Nuxt 4
└── nuxt.config.ts              # Configuration Nuxt
```

Annexe B : Extrait du schéma Prisma (modèle User)

```
model User {
  id                    String    @id @default(cuid())
  first_name            String
  last_name             String
  age                   Int
  email                 String    @unique
  phone                 String    @unique
  password_hash         String
  profile_picture_url   String?
  email_verified        Boolean   @default(false)
  email_verify_token    String?   // SHA-256 hash
  email_verify_expires  DateTime? // 24h TTL
  refresh_token         String?   // SHA-256 hash
  reset_password_token  String?   // SHA-256 hash
  reset_password_expires DateTime? // 1h TTL
  created_at            DateTime  @default(now())
  updated_at            DateTime  @updatedAt

  // Relations
  projects     ProjectMember[]
  bank_accounts BankAccount[]

  @@map("users")
}
```

Annexe C : Extrait du pipeline CI/CD (.github/workflows/ci.yml)

```yaml
name: Aligno CI Quality Gate
on:
  pull_request:
    branches: [main, dev]

jobs:
  validate:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: test_db
          POSTGRES_PASSWORD: password
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: npm ci
      - name: Generate Prisma Client & Nuxt Types
        run: npx prisma generate && npx nuxt prepare
      - name: Apply DB migrations
        run: npx prisma migrate deploy
      - name: TypeScript type check
        run: ./node_modules/.bin/vue-tsc --noEmit
      - name: Run full test suite
        run: npm run test
```

[Réservé — Figure 8] Capture d'écran du tableau Kanban sur Jira en fin de Sprint 3. Le tableau montre les tickets répartis en colonnes : Todo, In Progress, In Review, Done.

[Réservé — Figure 9] Capture d'écran du rapport Lighthouse SEO de la marketplace SikBeauty après migration Next.js — Score : 98/100.

[Réservé — Figure 10] Capture d'écran du pipeline GitHub Actions en cours d'exécution — Étapes : Install, Generate, Type-check, Test (35/35 passed).